

Դասեր (Class)

Համակարգչային գիտության ոլորտում գոյություն ունեն ծրագրավորման տարբեր հայեցակարգեր, որոնք նախատեսված են տարբեր խնդիրներ լուծելու համար: Մինչ class-ներին ացնելը՝ եկեք թվարկենք և սահմանենք հիմնական հայեցակարգերը:

- **Ֆունկցիոնալ ծրագրավորում (Functional programming)**

FP-ն կենտրոնանում է մաքուր մաթեմատիկական ֆունկցիաների (pure functions) և փոփոխականների սահմանափակման վրա: Այս տարբերակը հարմար է տարատեսակ մաթեմատիկական հաշվարկների և տվյալների վերլուծության համար: Ֆունկցիոնալ ծրագրավորում աջակցող լեզուներից են՝ Haskell, JavaScript, Python, Scala, Erlang, Lisp, ML, Clojure, OCaml, Common Lisp, Racket:

- **Ընթացակարգային ծրագրավորում (Procedural programming, PP)**

PP-ն կենտրոնացած է ֆունկցիաների և հրահանգների հաջորդականության վրա: Երբեմն անվանում են 3GL, որը նշանակում է երրորդ սերոնի ծրագրավորման լեզու: Այն պահանջում է, որ ծրագրավորողը կոդավորի ոչ միայն թե «*ինչ անել*», այլ նաև «*ինչպես անել*»: Հետևյալ ծրագրավորման լեզուները հիմնականում օգտագործել են այս հայեցակարգը՝ FORTRAN, C, Basic, Pascal, ALGOL, COBOL, JAVA: Նմանատիպ ծրագրավորումն այսօր գրեթե չի օգտագործվում, սակայն դեռևս մի շարք երկրներում պետական համակարգի ծրագրային ապահովումներն աշխատում են այս լեզուներով և մեթոդով գրված ծրագրերով:

- **Ռեակտիվ ծրագրավորում (Reactive programming, RP)**

RP-ն կենտրոնանում է իրադարձություններով կառավարվող ծրագրավորման վրա: Հարմար է UI, ապարատողության և իրական ժամանակի տվյալների մշակման համար: Աջակցող լեզուներից են՝ Haskell, Scala, Erlang, Elixir, Java, C# - ReactiveX:

- **Առարկայա-կողմնորոշված ծրագրավորում (Object-oriented programming, OOP)**
OOP-ն կենտրոնանում է առարկաների վրա, որոնք տվյալներ (ատրիբուտներ) և գործողություններ (մեթոդներ) են պարունակում: Հարմար է իրական աշխարհի մոդելավորման համար: Սա ամենաշատ օգտագործվող հայեցակարգն է և օգտագործվում է գրեթե բոլոր առաջադեմ ծրագրավորման լեզուներում (Java, C++, Python, C#):

Այսպիսով այս հայեցակարգերն ունեն մի շարք առավելություններ և թերություններ, սակայն դրանք հիմնականում տարբեր ժամանակաշրջաններում են զարգացել՝ փոխարինելով մեկը մյուսին: Փաստացի կարող ենք դիտարկել որպես տրամաբանական զարգացում: Աղյուսակով ցույց տանք նշած հայեցակարգերի իմաստաբանական տարբերությունները.

Functional	Procedural	Reactive	Object-oriented
Function	Procedure	Observable/Stream	Method
Persistent Data Str.	Record	Signal/Event	Object
Module/Namespace	Module	Observable Pipeline	Class
Function composition	Procedure call	Subscription	Message (Method call)
Immutable data	Global state	Time-varying state	Instance state
Recursion	Loops	Operators (map, filter)	Inherit./Polymorphism
Values	Variables	Behavior / Subject	Properties
Pure functions	Side effects	Asynchronous events	Encapsulation

Եկեք օրինակի վրա տեսնենք վերոնշյալ հայեցակարգերով գրված Python լեզվի ծրագրավորման կոդերի տարբերությունները, որոնք տպելուց նույն արդյունք են տալիս:

Ունենք խնձոր, որի բնօրինակը կանաչ է և կախված կարիքներից հարկավոր է օգտագործել այն տարբեր գույներով:



Functional Programming (FP)

```
Python
def transform_apple(apple):
    """ Վերադարձնում է նոր
    փոխակերպված խնձորն առանց
    բնօրինակը փոփոխելու: """
    new_apple = {
        "size": apple["size"] * 2,
        "color": "red" if
apple["color"] == "green" else
"yellow"
    }
    return new_apple

# բնօրինակ խնձորը
original_apple = {"size": 5, "color":
"green"}

# ձևափոխում
transformed_apple =
transform_apple(original_apple)

print("Original Apple:",
original_apple)
print("Transformed Apple:",
transformed_apple)
```

Procedural Programming (PP)

```
Python
# խնձորի սկզբնական վիճակը
apple = {"size": 5, "color":
"green"}

# քայլ-առ-քայլ ձևափոխում
print("Step 1: Picking the
apple...")
apple["size"] += 2
print("Step 2: Changing the
color...")
apple["color"] = "red"
print("Step 3: Transformation
complete!")

print("Transformed Apple:", apple)
```

Reactive Programming (RP)

```
Python
import time

def transform_apple():
    print("Waiting for event (Press
Enter to transform the apple)...")
    input() # իրադարձություններն
ըստ օգտատիրոջ
    apple["size"] *= 2
    apple["color"] = "red"
    print("The apple has
transformed:", apple)

# խնձորի սկզբնական վիճակը
apple = {"size": 5, "color":
"green"}

print("Original Apple:", apple)
transform_apple()
```

Object oriented Programming (OOP)

```
Python
class Apple:
    def __init__(self, size,
color):
        self.size = size
        self.color = color

    def transform(self):
        """խնձորի հատկությունները
փոփոխելու մեթոդ. """
        self.size *= 2
        self.color = "red"
        print(f"Transformed Apple
-> Size: {self.size}, Color: |
{self.color}")

# ստեղծել խնձորի օբյեկտ
apple = Apple(size=5,
color="green")

print(f"Original Apple -> Size:
{apple.size}, Color:
{apple.color}")
apple.transform()
```

Մաքուր ֆունկցիաներ

Քայլեր

Իրադարձությունների

Ատրիբուտներ ու մեթոդներ

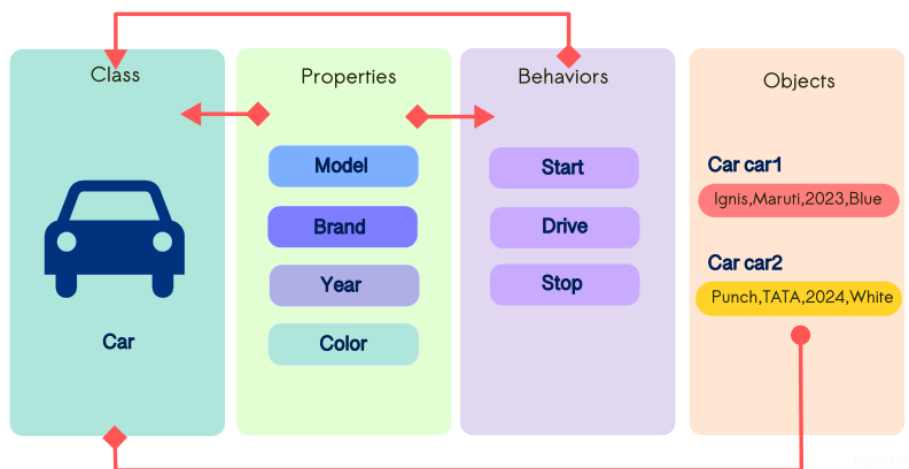
Original Apple -> Size: 5, Color: green

Transformed Apple -> Size: 10, Color: red

Ծրագրավորման սկզբնական փուլերում բոլոր գործողությունները իրականացվում էին ֆունկցիաների միջոցով: Ֆունկցիաները թույլ էին տալիս կոդը մասերի բաժանել, վերագտագործել և դառնում էին ավելի կառավարելի: Սակայն, երբ ծրագրերը մեծացան, առաջացան մի շարք խնդիրներ, որոնք ֆունկցիաներով հնարավոր չէր լուծել: Թվարկենք հիմնական խնդիրները.

1. Տվյալների և գործառույթների միջև կապի բացակայություն:
2. Ֆունկցիաների կոդմիջ տվյալները չէին պահպանվում:
3. Կոդի վերագտագործման սահմանափակում:
4. Կառավարման դժվարություն:

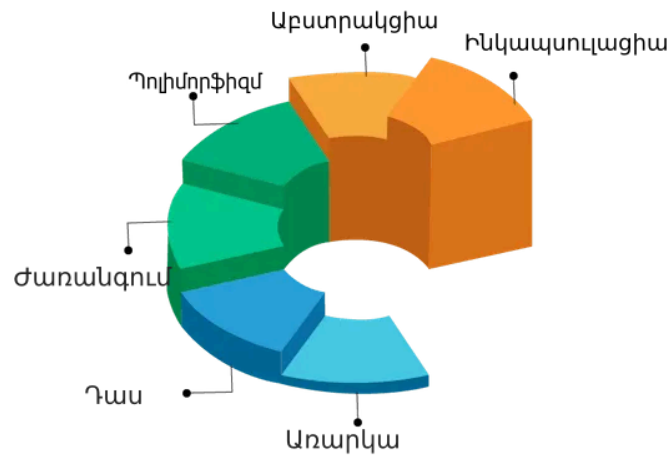
Այսպիսով կարիք եղավ ներդնելու ավելի բարձր կառուցվածքային տարր՝ **class (դաս)**: Այն առարկայի «կադապար» է, որը նկարագրում է հատկությունները (properties) և մեթոդները (methods):



Նկար 1. Class-ի (դաս) օրինակ մեքենայի համար: Աղբյուր՝ [bigkno!](#)

Python լեզվում և առհասարակ առարկայա-կոդմտորոշված ծրագրավորման (OOP) մեջ ժառանգականություն, պոլիմորֆիզմ և ինկապսուլացիա, աբստրակցիա հասկացությունները կարևոր դեր ունեն, քանի որ օգնում են կառուցել կազմակերպված, վերագտագործելի, համապարփակ և անվտանգ կոդեր: OOP-ն հիմնականում կենտրոնանում է առարկաների և class-ների վրա, սակայն այն ազդում

Է ամբողջ ծրագրի կառուցվածքի վրա: Ուստի, երբ ասում ենք OOP ավելի շատ նկատի ենք ունենում առարկան և class-ը:



Նկար 2. OOP ծրագրավորման բաղադրիչները: Աղբյուր՝ francescolelli.inf:

1. Ժառանգում (inheritance)

Ժառանգումը թույլ է տալիս մեկ դասին (դուստր կամ ենթադաս) ժառանգել մյուս դասից (ծնող դաս) հատկությունները (attributes) և մեթոդները (methods): Այսպիսով, նոր դասը կարող է օգտագործել նախապես գրված կոդը՝ առանց այն կրկնելու:

2. Պոլիմորֆիզմ (Polymorphism)

Բառի բացատրությունը.

Պոլի՝ շատ, մորֆ՝ ձև: Բազմաթիվ տվյալների ձևաչափում կամ կարգավորում՝ բազմաձևություն:

Պոլիմորֆիզմի կարիքը ծագեց 1960-ականներին՝ առարկայա-կողմնորոշված ծրագրավորման (OOP) զարգացման փուլում: Սկսեցին միևնույն մեթոդն օգտագործել տարբեր առարկաների համար՝ հեշտացնելով կոդի վերաօգտագործումն ու ընդլայնումը:

Հետևաբար, պոլիմորֆիզմը թույլ է տալիս, որ տարբեր դասեր (class) ունենան նույն անունով մեթոդներ, բայց կատարեն տարբեր իրագործումներ: Այսպիսով, մենք կարող ենք գրել ունիվերսալ կոդ, որը կաշխատի տարբեր օբյեկտների համար:



Նկար 3. Պոլիմորֆիզմի օրինակ իրական կյանքից: Աղբյուր՝ [vividexamples](http://vividexamples.com)

Նկար 1-ում պատկերված է թիթեռի զարգացման տարբեր փուլերի ձևափոխները՝ թրթուրային ձևից մինչև ամբողջական հասուն միջատ: Այս զարգացումը նման է class-ների պոլիմորֆիզմին, քանի որ բոլոր փուլերում այն թիթեռ է, սակայն արտաքին տեսքերը տարբերվում են:

Ինկապսուլացիա (Encapsulation)

Բառի բացատրությունը.

Capsule-«կապսուլ» կամ հայերեն պարկուճ բառն է հիմքում: Ընդհանուր բառը նշանակում է փակել, ծածկել կամ կաղապարել:



Նկար 4. Դեղահաբի պարկուճ, պատկերի աղբյուր՝ wiki

Ծրագրավորման լեզուներում (C, C++, Python) ինկապսուլացիան տվյալների պաշտպանման մեխանիզմ է, որը սահմանափակում է արտաքին միջավայրից

տվյալներին անմիջապես հասանելիությունը: Սա ապահովվում է private (մասնավոր) փոփոխականների և մեթոդների միջոցով (__ նախաձևացով):



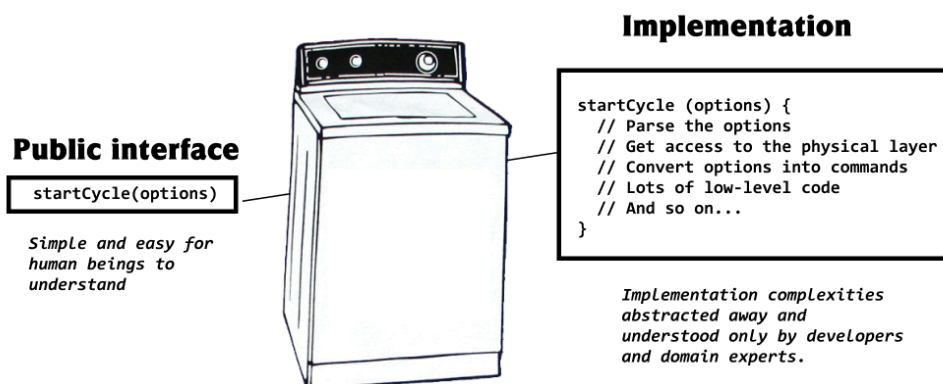
Նկար 5. Ինկապսուլիացիան ծրագրավորման մեջ, պատկերի աղբյուր՝ [usemynotes](#):

Աբստրակցիա (Abstraction)

Բառի բացատրությունը.

Բացառում, անջատում, վերացում, վերացողականություն:

Աբստրակցիայի գաղափարը սկսեց կարիք ունենալ այն ժամանակ, երբ ծրագրավորումը դարձավ ավելի բարդ և բազմազան (1960-1970 թվականներին): Մեծ ծրագրերում հարկավոր է լինում որոշ մասեր թաքցնել և օգտատերերին հասանելի դարձնել միայն անհրաժեշտության դեպքում:



Նկար 6. Աբստրակցիան իրական կյանքում, պատկերի աղբյուր՝ [khalilstemmler](#):

Խնդիրներ դասեր թեմայից (class)

1. Ստեղծել **Student** դասը (class):

Ատրիբուտներ	Մեթոդներ
- name ` անուն,</li><li>- <b>age</b> ` տարիք, - grade ` դասարան:	- introduce(), որը կտալի. "Ես [անուն] եմ, [տարիք] տարեկան եմ և սովորում եմ [դասարան]-ի դասարանում:"

2. Ստեղծել **Car** դասը:

Ատրիբուտներ	Մեթոդներ
- brand ` արտադրող,</li><li>- <b>model</b> ` մոդել, - year ` արտադրության տարեթիվ:	- car_info() ` վերադարձնում է մեքենայի արտադրողի և մոդի անվանումը,</li><li>- <b>age(current_year)</b> ` վերադարձնում է մեքենայի տարիքը կախված ընթացիկ տարվանից:

3. Ստեղծել **FootballPlayer** դասը:

Ատրիբուտներ	Մեթոդներ
- name ` խաղացողի անուն,</li><li>- <b>team</b> ` ցիաղացողի ներկա թիմը, - position ` խաղացողի դիրքը:	- player_info() ` տալում է խաղացողի տվյալները,</li><li>- <b>change_team (new_team)</b> ` փոխում է խաղացողի թիմը:

4. Ստեղծել **Book** դաս (class):

Ատրիբուտներ	Մեթոդներ
- title ` գրքի վերնագիր,</li> <li>- <b>author</b> ` հեղինակ, - genre ` ժանր,</li> <li>- <b>pages</b> ` էջերի քանակը, - publisher ` հրատարակչություն,</li> <li>- <b>publication_year</b> ` տպագրման տարեթիվ:	- get_description(): վերադարձնում է գրքի կարճ նկարագրությունը՝ վերնագրով և հեղինակով, - sheet_count(): վերադարձնում է գրքի թերթերի քանակը, - age(current_year): հաշվում է գրքի տարիքը՝ ելնելով տպագրման տարեթվից:

5. Ստեղծել **Animal** դասը:

Ատրիբուտներ	Մեթոդներ
- age ` տարիք,</li> <li>- <b>weight</b> ` քաշ, - speed ` արագություն:	- info() ` վերադարձնում է կենդանու բոլոր տվյալները,</li> <li>- <b>update_speed(n)</b> ` կենդանու արագությունը դարձնում է n, - change_weight (a) ` փոխում է կենդանու քաշը a-ով:

- Ստեղծել **Cat** դասը, որը **կժառանգի** animal դասից:

Նոր ատրիբուտներ	Նոր մեթոդներ
-----------------	--------------

- <code>color</code> `գույն,</li> <li>- <code>species</code> `տեսակ (օրինակ `siamese, bengal):	- <code>get_age_category()</code> `գտնում և տալում է. ☐ կատուն երիտասարդ է, տարիքը փոքր է 7-ից, ☐ միջին տարիքի է, 7-ից 11 տարեկան, ☐ ծեր է, 11-ից բարձր:
--	--

6. Ստեղծել `Car` դասը:

Ատրիբուտներ	Մեթոդներ
- <code>brend</code> `արտադրող,</li> <li>- <code>model</code> `մոդել, - <code>year</code> `արտադրման տարեթիվ,</li> <li>- <code>max_speed</code> `առավելագույն արագությունը կմ/ժ-ով, - <code>color</code> `գույն:	- <code>info()</code> `տալում է մեքենայի բոլոր տվյալները,</li> <li>- <code>change_color (color)</code> `փոխում է մեքենայի գույնը: - <code>age (current_year)</code> `հաշվում է մեքենայի տարիքը:

- Ստեղծել `Elector_car` ենթադասը, որը **ժառանգում է** `Car` դասից:

Նոր ատրիբուտներ	Նոր մեթոդ
- <code>battery_size</code> `մարտկոցի չափը կՎտժ, - <code>charging_time</code>: լիցքավորման ժամանակը (րոպե):	- <code>price()</code> `հաշվում է ամբողջական լիցքավորման արժեքը (1 կՎտ.ժ-ի արժեքը 120 դրամ):

- Ստեղծել `Hydrogen_car` ենթադասը, որը **ժառանգում է** `Car` դասից:

Նոր ատրիբուտներ	Նոր մեթոդ
-----------------	-----------

- <code>hydrogen_tank_capacity</code>՝մեքենայի բակի տարողությունը (կգ), - <code>current_hydrogen_level</code>՝ մեքենայի ընթացիկ վառելիքի ծավալը, որը պետք է փոքր կամ հավասար լինի բակի տարողությունից, - <code>range_per_kg</code>՝ քանի կմ կարող է անցնել 1 կգ ջրածնով:	- <code>refuel(amount)</code>՝ լրացնում է ջրածնի բաքը amount կգ-ով, բայց ոչ ավել քան <code>hydrogen_tank_capacity</code>-ն: - <code>calculate_range()</code>՝ հաշվարկում է, թե քանի կմ կարող է անցնել ներկա ջրածնի մակարդակով, - <code>check_fuel_status()</code>՝ վերադարձնում է, թե քանի % ջրածին է մնացել բաքում:
--	--

7. Ստեղծել `Animal` դասը:

Ատրիբուտներ	Մեթոդներ
- <code>age</code>՝ տարիք, - <code>weight</code>՝ քաշ, - <code>speed</code>՝ արագություն:	- <code>info()</code>՝ վերադարձնում է կենդանու բոլոր տվյալները: - <code>update_speed(n)</code>՝ կենդանու արագությունը դարձնում է <code>n</code>: - <code>change_weight(a)</code>՝ փոխում է կենդանու քաշը <code>a</code>-ով: - <code>make_sound()</code>՝ վերադարձնում է <code>"unknown"</code> տեքստ:

- Ստեղծել `Cat`, `Dog` և `Cow` ենթադասեր, որնք ժառանգում են `Animal` դասից, իսկ `make_sound()` մեթոդը վերադարձնում է հետևյալ ձայները (պոլիմորֆիզմ):

Dog - "Woof-woof"

Cat - "Meow"

Cow - "Moo"

- Ստեղծել `Fish` ենթադասը, որը ժառանգում է `Animal` դասից, սակայն չի ունենում `make_sound()` մեթոդը:

8. Ստեղծել **CreditCardPayment** դաս:

Ատրիբուտներ	Մեթոդներ
- card_number ` քարտի համար,</li><li>- <b>card_holder</b> ` քարտապանի անունը, - balance ` գումարի մնացորդը:	- pay(amount) ` կատարում է վճարում, եթե հաշվի մնացորդը թույլ է տալիս և տպում է (<i>պոլիմորֆիզմ</i>). <i>Paid \$50 using Credit Card (1234-5678-9876-5432).</i>

- Ստեղծել **PayPalPayment** դասը:

Ատրիբուտներ	Մեթոդներ
- email ` օգտատիրոջ PayPal էլ. փոստը,</li><li>- <b>balance</b> ` գումարի մնացորդ:	- pay(amount) ` կատարում է վճարում, եթե հաշվի մնացորդը թույլ է տալիս և տպում է . <i>Paid \$30 using PayPal (john@example.com):</i>

9. Ստեղծել **Square** դասը:

Ատրիբուտ	Մեթոդներ
- length ` կողմի երկարություն:	- area() ` հաշվարկում է քառակուսու մակերեսը,</li><li>- <b>perimeter()</b> ` հաշվարկում է քառակուսու պարագիծը:

- Ստեղծել **Triangle** դասը:

Ատրիբուտներ	Մեթոդներ
-------------	----------

- `side1` ` եռանկյան առաջին կողմ,
- `side2` ` եռանկյան երկրորդ կողմ,
- `side3` ` եռանկյան երրորդ կողմ:

- `is_void()` ` ստուգում է նշված կողմերով եռանկյուն կա, թե ոչ
- `area()` ` հաշվում է եռանկյան մակերեսը,
- `perimeter()` ` հաշվարկում է եռանկյան պարագիծը:

- Ստեղծել `Circle` դասը:

Ատրիբուտ	Մեթոդներ
- <code>radius</code> ` շառավիղ:	- <code>circumference()</code> ` շրջանագծի երկարություն,</li> <li>- <code>area()</code> ` շրջանի մակերես:

10. Ստեղծել `Person` դասը:

Ատրիբուտներ	Մեթոդներ
- <code>name</code> ` անուն (private),</li> <li>- <code>age</code> ` տարիք (private), - <code>id</code> ` անձնագրի համար (private):	- <code>get_info()</code> ` վերադարձնում է անունը, տարիքը,</li> <li>- <code>get_older()</code> ` տարիքը մեծացնում է 1-ով, - <code>get_id()</code> ` վերադարձնում է անձնագրի համարը:

11. Ստեղծել `BankAccount` դասը (*ինկապսուլացիա*):

Ատրիբուտներ	Մեթոդներ
-------------	----------

- `balance` `հաշվի մնացորդ (private),
- `password` `գաղտնաբառ (private):

- `deposit(amount, password)` `ավելացնել գումար հաշվին: Գումարը պետք է դրական լինի, իսկ գաղտնաբառը՝ ճիշտ,
- `withdraw(amount, password)` `հանել գումար հաշվից, amount-ը չպետք է հաշվի մնացորդից մեծ լինի, իսկ գաղտնաբառը պետք է ճիշտ լինի,
- `change_password(old_password, new_password)` `փոխում է գաղտնաբառը, եթե հին գաղտնաբառը ճիշտ է գրված,
- `get_balance(password)` `վերադարձնում է հաշվի մնացորդը, եթե գաղտնաբառը ճիշտ է:

12. Ստեղծել `WeatherStation` դասը:

Ատրիբուտներ

- `humidity` `խոնավություն (private),
- `pressure`: `մթնոլորտային ճնշում (private),
- `temperature` `ջերմաստիճան (private):

Մեթոդներ

- `get_humidity()` `վերադարձնում է խոնավությունը,
- `get_pressure()` `վերադարձնում է մթնոլորտային ճնշումը,
- `get_temperature()` `վերադարձնում է ջերմաստիճանը,
- `update_measure(tem,hum,pre)` `թարմացնում է ջերմաստիճանի, խոնավության և ճնշման տվյալները,
- `average_temperature()` `վերադարձնում է միջին ջերմաստիճանը,
- `average_humidity()` `վերադարձնում է միջին խոնավությունը,
- `average_pressure()` `վերադարձնում է միջին մթնոլորտային ճնշումը,

- `max_temperature()`, `min_temperature()` ` վերադարձնում է առավելագույն / նվազագույն ջերմաստիճանները,
- `max_humidity()`, `min_humidity()` ` վերադարձնում է առավելագույն / նվազագույն խոնավությունը,
- `max_pressure()`, `min_pressure()` ` վերադարձնում է առավելագույն, նվազագույն մթնոլորտային ճնշումը,
- `draw_scatter()` ` կազմում է ցրված (scatter plot) դիագրամ ջերմաստիճանի կախումը խոնավությունից,
- `last_5_measurements()` ` վերադարձնում է վերջին 5 չափումների տվյալները: Եթե տվյալները քիչ են 5-ից, ապա կտալի բոլոր տվյալները:

13. Ստեղծել `Library` դասը:

Ատրիբուտ	Մեթոդներ
- <code>book</code> ` բառարաններից կազմված ցուցակ է ` <code>{title: str, author: str, available: bool}</code> (private):	- <code>add_book(title, author)</code> ` ավելացնում է նոր գիրք գրադարանի բազայում,</li> <li>- <code>remove_book(title)</code> ` հեռացնում է գիրքը գրադարանից, - <code>borrow_book(title)</code> ` նշում է, որ գիրքը վերցրած է և հասանելիությունը դառնում է False,</li> <li>- <code>return_book(title)</code> ` նշում է, որ գիրքը վերադարձվել է գրադարան, - <code>list_available_books()</code> ` վերադարձնում է բոլոր հասանելի գրքերը,</li> <li>- <code>search_by_author(author)</code> ` վերադարձնում է տվյալ հեղինակին պատկանող բոլոր գրքերը, - <code>save_to_file(file_name)</code> ` ստեղծում է <code>file_name</code> անունով txt ֆայլ և վրան պահում է գրքերի ցանկը:

14. Ստեղծել `GPS_Tracker` դաս:

Ատրիբուտներ	Մեթոդներ
- <code>latitude</code>՝ լայնություն (private), - <code>longitude</code>՝ երկայնություն (private):	- <code>move(new_latitude, new_longitude)</code>՝ փոխում է օբյեկտի դիրքը, - <code>current_location()</code>՝ վերադարձնում է ընթացիկ դիրքի կոորդինատները: - <code>distance()</code>՝ վերադարձնում է օբյեկտի վերջին տեղափոխության երկարությունը, - <code>all_location()</code>՝ վերադարձնում է օբյեկտի եղած բոլոր դիրքերի կոորդինատները, - <code>distance_from_origin()</code>՝ վերադարձնում է սկսագնակետի և ներկա դիրքի հեռավորությունը, - <code>higher()</code>՝ վերադարձնում է ներկա դիրքի բարձրությունը ծովի մակարդակից:

Python

```
import requests
```

```
def get_elevation(latitude, longitude):
    # Օգտագործում ենք OpenElevation API
    url =
    f"https://api.open-elevation.com/api/v1/lookup?locations={latitude},{longitude}"

    response = requests.get(url)

    if response.status_code == 200:
        data = response.json()
        elevation = data['results'][0]['elevation']
        return elevation
```



```

else:
    return "Error: Could not retrieve elevation data."

```

```

# GPS կոորդինատների օրինակ

```

```

latitude = 28.596111 # Աշխարհագրական լայնություն

```

```

longitude = 83.820274 # Աշխարհագրական երկայնություն

```

```

elevation = get_elevation(latitude, longitude)

```

```

print(f"Բարձրությունը ծովի մակարդակից՝ {elevation} մետր:") #ուկի
որոշակի սխալանքի չափ

```

15. Ստեղծել **HEV** դաս ([Hybrid Electric Vehicle](#)):

Ատրիբուտներ	Մեթոդներ
- brand՝ արտադրող, - model՝ մոդել, - fuel_efficiency՝ վառելիքի արդյունավետությունը (Էլեկտրաէներգիայի հետ միաժամանակ 100 կմ-ի համար օգտագործվող վառելիքի քանակը) - battery_level՝ մարտկոցի մակարդակ (0-100%): - fuel_capacity՝ վառելիքի քանակը (առավելագույնը 50լ):	- info ()՝ վերադարձնում է մեքենայի բոլոր տվյալները, - refuel (amount): ավելացնում է վառելիքի ծավալը amount-ով, եթե լիցքավորումից հետո, այն չի գերազանցում 50 լիտրը:

- Ստեղծել **PHEV** ենթադասը ([Plug-in Hybrid Electric Vehicle](#)):

Ատրիբուտներ*	Նոր մեթոդներ
- Կժառանգի HEV դասից, բացառությամբ fuel_efficiency ատրիբուտի:	- fuel_cost ` ցույց կտա առանց էլեկտրաէներգիայի բենզինի վառելիքի ծախսը 100 կմ-ի համար,</li> <li>- <b>charge(procent)</b> ` լիցքավորում է մեքենան համապատասխան տոկոսով:

- Ստեղծել **BEV** ենթադասը: (Battery Electric Vehicle): Այս դասը կժառանգի PHEV դասից, բացառությամբ **fuel_capacity**, **fuel_cost** ատրիբուտներից և **refuel()** մեթոդից:

16. Ստեղծել **SmartPhone** դասը:

Ատրիբուտներ	Մեթոդներ
- brand ` արտադրող,</li> <li>- <b>model</b> ` մոդել, - price ` գինը,</li> <li>- <b>color</b> ` գույնը:	- make_call(number) ` տպում է հետևյալ հաղորդագրությունը. <i>“Զանգ է կատարվում [number] համարին”</i></li> <li>- <b>send_message(number, message)</b> ` տպում է հետևյալ հաղորդագրությունը. <i>“Հաղորդագրություն [message] ուղարկել է [number] համարին”</i>

- Ստեղծել **AndroidPhone** ենթադասը:

Լրացուցիչ ատրիբուտ	Լրացուցիչ մեթոդ
- <code>android_version</code> ` android-ի տարբերակ:	- <code>install_app(app_name)</code> ` տպում է հետևյալ հաղորդագրությունը. <i>“[app_name] հավելվածը ներբեռնված է”</i>

- Ստեղծել `Iphone` ենթադասը:

Լրացուցիչ ատրիբուտ	Լրացուցիչ մեթոդ
- <code>ios_version</code> ` iOS-ի տարբերակը:	- <code>use_siri(command)</code> ` տպում է հետևյալ հաղորդագրությունը. <i>“siri-ն կատարում է [command] հրամանը”</i>

- Ստեղծել `GamingPhone` ենթադասը:

Լրացուցիչ ատրիբուտներ	Լրացուցիչ մեթոդներ
- <code>cooling_system</code> ` True կամ False, արդյո՞ք ունի հովացման համակարգ,</li> <li>- <code>game_mode</code> ` սկզբում լռելայն ընդունում է False արժեքը:	- <code>change_mode ()</code> ` եթե խաղային ռեժիմը միացած է, անջատում է, եթե անջատած է միացնում է: Այսինքն <code>game_mode</code>-ը փոխում է հակառակ արժեքի:</li> <li>- <code>boost ()</code> ` կախված <code>game_mode</code>-ի արժեքից տպում է <i>“խաղային ռեժիմը միացված է”</i> կամ <i>“խաղային ռեժիմը անջատված է”</i> հաղորդագրություններից մեկը:

17. Ստեղծել **GraphPlotter** դասը:

Ատրիբուտներ	Մեթոդներ
- title ` վերնագիր,</li><li>- <b>x_data</b> ` սկզբում դատարկ ցուցակ է, - y_data ` սկզբում դատարկ ցուցակ է,</li><li>- <b>color</b> ` գույն:	- add_point (x,y) ` ավելացնում է նոր կետ,</li><li>- <b>avg_data()</b> ` վերադարձնում է x և y տվյալների միջինը, - scatter_plot () ` գծում է x, y տվյալների ցրված գրաֆիկը,</li><li>- <b>change_color(new_color)</b> ` փոխում է գույնը, - save_scatter_plot(path) ` պահպանում է scatter plot-ը որպես նկար նշված ուղիով:

18. Ստեղծել **TemperatureAnalyzer** դասը:

Ատրիբուտներ

- **days** ` օրեր, սկզբում numpy-ի դատարկ զանգված է,
- **temperatures** ` ջերմաստիճաններ սկզբում numpy-ի դատարկ զանգված է,
- **unit** ` ջերմաստիճանի միավորը (“°C” կամ “°F”):

Մեթոդներ

- **load_data_from_file(filename)** ` ներմուծում է օրերի և ջերմաստիճանների տվյալները csv ֆայլից,
- **temperature_scatter_plot ()** ` տպում է ցրված գրաֆիկը,
- **hottest_day()** ` վերադարձնում է գրանցված ամենաբարձր ջերմաստիճանը,
- **coldest_day()** ` վերադարձնում է գրանցված ամենաացածր ջերմաստիճանը,
- **temperature_histogram()** ` կառուցում է ջերմաստիճանների հիստոգրամը,
- **temperature_mean()** ` հաշվում է միջինը,
- **temperature_median()** ` հաշվում է մեդիանը,

- `temperature_var()` `հաշվում է վարիացիան,
- `temperature_std()` `հաշվում է ստանդարտ շեղումը,
- `convert_to_fahrenheit()` `ջերմաստիճանները temperature զանգվածում փոխակերպում է ֆարենհայտի,
- `convert_to_celsius()` `ջերմաստիճանները temperature զանգվածում փոխակերպում է ցելսիուսի:

19. Ստեղծեք `Runner` դասը:

Ատրիբուտներ

- `name` `պահում է մարզիկի անունը:

Մեթոդներ

- `add_run(self, day, time)` `ավելացնում է մարզման օրը (`day`) և վազքի ժամանակը:
- `run_line_plot(self, color)` `line plot-ով պատկերում է մարզիկի ցույց տված արդյունքները,
- `run_time_mean(self)` `հաշվում է վազքի միջին ժամանակը:
- `best_run(self)` `գտնում է լավագույն արդյունքը (ամենաքիչ ժամանակով վազքը):

20. Ստեղծել արևոտրակտ `Shape` class-ը, որը կունենա արևոտրակտ `area()` և `perimeter()` մեթոդները:

Այնուհետև ստեղծել հետևյալ դասերը.

- `Rectangle`, որի `area()` մեթոդը կվերադարձնի մակերեսը, իսկ `perimeter()` մեթոդը` պարագիծը:
- `Trinagle`, որի `area()` մեթոդը կվերադարձնի մակերեսը, իսկ `perimeter()` մեթոդը` շրջանագծի երկարությունը:

Python

```
from abc import ABC, abstractmethod
import math
```

```
# Արևոտրակտ class
```

```

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

# Ուղղանկյուն class
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

# Եռանկյուն class
class Trinagle (Shape):
    def __init__(self, side1,side2,side3):
        self.side1 = side1
        self.side2 = side2
        self.side3 = side3

```